

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	Prena-M	Reg. No.:	192524056
Section / Batch:	Btech AIGPS 1st year.	Date:	28/08/2026

Code of Conduct

I Prena-M [192524056] (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Prena-M

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

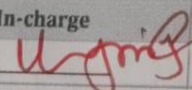
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

16 Linear probing vs separate chaining

Factor	Linear Probing	Separate chaining
clustering effect	suffers from primary clustering. consecutive occupied slots form long clusters	no primary clustering because collisions are stored in separate linked lists/buckets
Memory usage	uses the hash table array itself, so it has low memory overhead	Requires extra memory for linked-list nodes/pointers
performance under high factor	performance decreases rapidly as the table becomes full. searching & insertion take more time	Generally performs better at high load factors because multiple elements can share a bucket.
Large scale suitability	suitable when memory efficiency and cache performance are important, but requires a sufficiently large table	more suitable for large-scale applications because it handles collisions and high load factors more effectively

conclusion:

Therefore, separate chaining is generally preferred for large-scale applications

2/ Quick sort vs Merge sort for a large linked list (10 million nodes)

Factor	quick sort	Merge sort
Memory requirement	usually in-place, but partition needs stack space	Requires extra space for merging; for linked lists, merging can be done with very little extra node storage
Cache behaviour	Better for arrays because elements are accessed continuously; poor for linked lists due to non-contiguous access	very suitable for linked list because nodes can be rearranged using pointers without random access
Time complexity	Average: $O(n \log n)$; worst: $O(n^2)$	Best Average worst: $O(n \log n)$
Stability	Generally not stable	Stable sorting algorithm
Linked-list suitability	Less suitable because partitioning requires traversing nodes	Highly suitable because splitting & merging linked lists are efficient

What about arrays

For arrays, the choice can change. Quick sort is often preferred because it is in place, has good cache locality, and usually performs very fast in practice.

conclusion:

Linked list $\rightarrow$ Merge sort

Quadratic probing vs Double Hashing

Factor	Quadratic probing	Double hashing
clustering issues	Reduces primary clustering, but may suffer from secondary clustering	Minimizes both primary and secondary clustering because each key has a different probe sequence.
collision resolution efficiency	More efficient than linear probing, but probe positions follow a fixed quadratic pattern	Generally more efficient because the second hash function gives a better distribution of probes
suitability for dense hash tables	Less suitable when the table is very dense because collisions increase as the load factor becomes high	More suitable for dense hash tables because it distributes elements more uniformly and reduces clustering

conclusion:

quadratic probing is simple and reduces primary clustering, but Double Hashing provides better collision handling and more uniform distribution. Therefore, Double Hashing is generally preferred for dense hash tables

Best-case vs worst-case time complexity

Sorting Algorithms

Aspect	Best case	worst case
time complexity	usually $O(n \log n)$ for efficient algorithms like merge sort & quick sort	can be $O(n^2)$ for algorithms like quick sort with poor pivot selection
practical significance	Represents the situation where the input is favorable and the algorithm performs efficiently	Important because it shows the maximum time an algorithm may take for unfavorable input
Impact on algorithm selection	useful when inputs are usually favorable & performance	Important when predictable performance & reliability are required.

Examples:

- * Merge sort = Best = $O(n \log n)$, worst = $O(n \log n)$
- * Quick Sort = Best = $O(n \log n)$, worst = $O(n^2)$
- * Insertion sort = Best = $O(n)$, worst = $O(n^2)$

Conclusion:

Best-case complexity shows the algorithm's maximum efficiency, while worst-case complexity shows its performance limit.

Q6 Stable vs unstable sorting

Factor	Stable sorting	Unstable sorting
Definition	Maintains the original order of equal elements	May change the order of equal elements
Examples	Merge sort, Insertion sort	Quick sort, selection sort
order preservation	Equal elements remain in their original order	Equal elements may be rearranged
practical use	useful for sorting records based on multiple fields	Suitable when stability is not required.

Conclusion:

Stable sorting is preferred when the relative order of equal elements must be preserved.

Uppu